

Math Camp 2026

Calculus • Linear algebra • Data wrangling

Zhilin Lu

2026-08-14

Today's roadmap

Part I • Calculus

Derivatives and the rules → optimization by FOC and SOC → partial derivatives and the gradient → just enough integration to read a density

Part II • Linear algebra

Vectors and matrices → multiplication, conformability, inverses → reading $y = X\beta + \varepsilon$ line by line

Part III • Data wrangling

Reading and inspecting files → the pipe and the five verbs → grouping and joins → tidy data and pivoting → missing values → a full exercise

One question ties the three together: how does a table of numbers become an estimate?

Part I: Calculus

Maximum likelihood is an optimization problem. Optimization is calculus.

Almost every estimator you meet in the first year is defined as “the value of θ that maximizes (or minimizes) something.” Today is how you find it.

- Derivatives tell you which way is uphill
- Second derivatives tell you whether you are at a peak or a valley
- Partial derivatives handle more than one parameter at a time

Derivatives as slopes

The average rate of change between two points is a slope. Shrink the gap:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- $f'(x)$ is the slope of the tangent line at x
- Read it as: the instantaneous rate of change of f at x
- Notation: $f'(x)$, $\frac{dy}{dx}$, $\frac{d}{dx} f(x)$ — all the same object

This is yesterday's limit doing real work.

The basic rules

$$\frac{d}{dx}c = 0 \quad \frac{d}{dx}x^n = nx^{n-1} \quad \frac{d}{dx}[cf(x)] = cf'(x)$$

$$\frac{d}{dx}[f(x) \pm g(x)] = f'(x) \pm g'(x)$$

Quick ones:

$f(x)$	$f'(x)$
x^3	$3x^2$
$5x^2 - 2x + 7$	$10x - 2$
$\sqrt{x} = x^{1/2}$	$\frac{1}{2}x^{-1/2}$
$1/x = x^{-1}$	$-x^{-2}$

$$[fg]' = f'g + fg'$$

$$\left[\frac{f}{g}\right]' = \frac{f'g - fg'}{g^2}$$

Example: $f(x) = x^2 e^x$

$$f'(x) = 2xe^x + x^2 e^x = e^x(2x + x^2)$$

$$\frac{d}{dx}e^x = e^x \quad \frac{d}{dx}\ln x = \frac{1}{x}$$

With the chain rule:

$$\frac{d}{dx}e^{g(x)} = g'(x)e^{g(x)} \quad \frac{d}{dx}\ln g(x) = \frac{g'(x)}{g(x)}$$

That second form is why log-likelihoods are tractable: the log of a density differentiates into a ratio, and the ratio usually simplifies.

For composite functions, differentiate outside then inside:

$$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$$

$$\frac{d}{dx}e^{2x} = 2e^{2x} \quad \frac{d}{dx}\ln(3x^2) = \frac{6x}{3x^2} = \frac{2}{x}$$

This is the rule you will use most. Log-likelihoods are composite functions almost by construction.

$f''(x)$ is the derivative of the derivative — the rate at which the slope is changing.

- $f''(x) > 0$: convex, curving up, holds water
- $f''(x) < 0$: concave, curving down, spills water
- $f''(x) = 0$: possible inflection point

A concave log-likelihood is a well-behaved one: a single peak, and any hill-climbing algorithm finds it.

At an interior maximum or minimum, the tangent is flat:

$$f'(x^*) = 0$$

Solve for x^* : these are the **critical points**.

Example: $f(x) = -x^2 + 4x + 1$

$$f'(x) = -2x + 4 = 0 \implies x^* = 2$$

The FOC finds candidates. It does not tell you which kind you found.

Optimization: second-order condition

Evaluate f'' at the critical point:

- $f''(x^*) < 0 \rightarrow$ local **maximum**
- $f''(x^*) > 0 \rightarrow$ local **minimum**
- $f''(x^*) = 0 \rightarrow$ inconclusive, look further

Continuing the example: $f''(x) = -2 < 0$, so $x^* = 2$ is a maximum.

When you report a standard error from an MLE, you are reading curvature off f'' — sharper peak, more information, smaller SE.

With several variables, differentiate with respect to one and hold the rest fixed.

$$f(x, y) = x^2y + 3y^2$$

$$\frac{\partial f}{\partial x} = 2xy \quad \frac{\partial f}{\partial y} = x^2 + 6y$$

The ∂ symbol signals “others held constant” — the same *ceteris paribus* move you make verbally when interpreting a regression coefficient.

Integration accumulates. The definite integral is the area under a curve:

$$\int_a^b f(x) dx$$

$$\int x^n dx = \frac{x^{n+1}}{n+1} + C \quad (n \neq -1) \quad \int \frac{1}{x} dx = \ln |x| + C$$

Fundamental theorem: differentiation and integration undo each other.

Why you need integration

Two facts you will use constantly on Day 3:

$$\int_{-\infty}^{\infty} f(x) dx = 1 \quad E[X] = \int_{-\infty}^{\infty} x f(x) dx$$

A density integrates to one. An expectation is a weighted average with the density as weights.

$$P(a \leq X \leq b) = \int_a^b f(x) dx$$

You will rarely compute these by hand. You do need to read them.

Part II — Linear algebra

You need to read $y = X\beta + \varepsilon$ and know what every symbol is.

- Data arrives as a rectangle: rows are units, columns are variables
- Matrix notation writes a model with 40 covariates as compactly as one with 2
- The regression estimator is one line of matrix algebra

Vectors

A vector is an ordered list of numbers.

$$\mathbf{a} = \begin{bmatrix} 2 \\ 5 \\ 1 \end{bmatrix}$$

- Addition is element-wise, and requires the same length
- Scalar multiplication scales every element
- Dot product: $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$ — a single number

The dot product is how a row of data meets a vector of coefficients.

A matrix is a rectangular array. Dimensions are always **rows × columns**.

$$X = \begin{bmatrix} 1 & 3 \\ 1 & 5 \\ 1 & 2 \end{bmatrix} \quad (3 \times 2)$$

- x_{ij} is the entry in row i , column j
- The **transpose** X' (or $X^\top$) flips rows and columns, giving 2×3
- A column of ones is the intercept

$$(A_{m \times n})(B_{n \times p}) = C_{m \times p}$$

The inner dimensions must match — this is **conformability**. Entry c_{ij} is the dot product of row i of A with column j of B .

- $AB \neq BA$ in general. Order is not negotiable.
- $(AB)' = B' A'$ — the transpose reverses the order

If a matrix expression in a paper looks strange, check conformability first. It usually explains the notation.

The identity matrix I has ones on the diagonal, zeros elsewhere:

$$AI = IA = A$$

The inverse A^{-1} satisfies $AA^{-1} = I$.

- Only square matrices can have inverses
- Singularity in practice = perfect collinearity in your covariates

That is exactly the error R throws when you include a dummy for every category.

$$\begin{array}{r} 2x + y = 5 \\ x - y = 1 \end{array} \iff A\mathbf{x} = \mathbf{b}$$

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$$

Solution: $\mathbf{x} = A^{-1}\mathbf{b}$, when the inverse exists.

Symbol	Dimension	What it is
y	$n \times 1$	outcome, one row per unit
X	$n \times k$	design matrix, column of 1s plus covariates
β	$k \times 1$	coefficients, what you estimate
ε	$n \times 1$	errors

And the estimator, which you will derive later but can now parse:

$$\hat{\beta} = (X'X)^{-1}X'y$$

$(X'X)^{-1}$ exists only if no column of X is a linear combination of the others.

You should now be able to:

- Differentiate a polynomial, an exponential, and a log
- Find a maximum with the FOC and verify it with the SOC
- Say what a partial derivative holds constant
- Check whether two matrices can be multiplied
- Name every object in $y = X\beta + \varepsilon$ and give its dimensions

Part III · Data wrangling

The unglamorous truth

You will spend far more time getting data into shape than running models. Doing it in code — rather than by hand in Excel — is what makes the result reproducible.

Today: read a file, inspect it, fix it, reshape it, join it, and end with an analysis-ready table.

```
library(tidyverse)
```

```
install.packages("tidyverse")  # once, ever  
library(tidyverse)             # every new session
```

- Installing downloads it to your computer. Loading makes it available *now*.
- A startup message listing conflicts is normal, not an error
- `dplyr::filter()` says explicitly which package's `filter()` you mean

Takeaway

could not find function "read_csv" almost always means you forgot `library(tidyverse)`.

Projects and paths

```
math-camp/  
├─ math-camp.Rproj  
├─ data/  
│   ├─ raw/          <- never edited by hand, never overwritten  
│   └─ clean/        <- produced by your scripts  
├─ R/  
│   └─ 01-clean.R  
└─ output/
```

- Open the project file, not the script. That sets the working directory for you.
- Then every path is relative: "data/raw/wdi.csv"
- Never `setwd("/Users/zhilin/Desktop/...")` — it breaks on every other machine

Takeaway

Raw data is read-only. Every change to it happens in a script, so the path from download to analysis is fully documented.

Reading files

```
readr::read_csv("data/raw/polity.csv")      # .csv
haven::read_dta("data/raw/wdi.dta")         # Stata
readxl::read_excel("data/raw/trade.xlsx")   # Excel
readr::read_rds("data/clean/panel.rds")     # R's own format
```

Two arguments worth knowing on day one:

```
read_csv(
  "data/raw/wdi.csv",
  na = c("", "NA", "..", "-99"), # treat these as missing on the way in
  col_types = cols(iso3 = col_character(), year = col_integer())
)
```

Takeaway

Always read the column specification R prints. It is telling you what it guessed.

What comes back: a tibble

```
# A tibble: 5,320 × 6
```

```
  iso3   year   gdp    pop region      democracy
  <chr> <int> <dbl>  <dbl> <chr>      <lgl>
1 USA   1990  5963.   249. Americas    TRUE
```

Type	Means	Watch for
<chr>	text	numbers stored as text
<dbl>	number with decimals	
<int>	whole number	
<lgl>	TRUE / FALSE	
<fct>	category with fixed levels	silent level ordering
<date>	a real date	dates read in as <chr>

```
glimpse(df)      # every column, its type, first values  
summary(df)      # min / median / max / NAs per column  
count(df, region) # how many rows per category  
nrow(df); ncol(df); names(df)
```

Run all of these *before* touching anything. Look specifically for:

- A column typed chr that should be numeric — usually one stray footnote character
- Missing-value codes like -99 or 999 masquerading as data
- count() showing "USA", "U.S.A.", and "United States" as three countries
- Row counts that do not match what the codebook promised

Reading an error message

```
Error in `filter()`:  
! object 'yaer' not found
```

Read it backwards: the **!** **line** is the problem, the top line is where it happened.

What you see	What it means
object 'x' not found	typo, or you never ran the line that created it
could not find function "f"	package not loaded
unused argument (foo = 1)	that function has no argument by that name
non-numeric argument to binary operator	you did math on a chr column
a + in the console	unfinished line — press Esc

Takeaway

A **warning** means it ran and you should look. An **error** means it did not run.

The pipe

```
df |> filter(year > 2000) |> select(country, exports)
```

Read |> as “then.” The object on the left becomes the first argument on the right.

```
# these are identical  
summarize(group_by(filter(df, year > 2000), region), m = mean(gdp))
```

```
df |>  
  filter(year > 2000) |>  
  group_by(region) |>  
  summarize(m = mean(gdp))
```

filter(): keeping rows

```
df |> filter(year >= 2010, region == "Asia")
df |> filter(region %in% c("Asia", "Europe"))
df |> filter(between(year, 1990, 2000))
df |> filter(is.na(gdp))
df |> filter(gdp > 1000 | pop > 50)
```

Operator	Means
== !=	equal to / not equal to
> >= < <=	comparisons
%in%	is one of these values
& !	and / or / not

Takeaway

= assigns, == compares. And `filter()` silently drops NA rows — that is a decision, so make it on purpose.

select() and rename(): columns

```
df |> select(country, year, gdp)           # keep these
df |> select(-notes, -source)             # drop these
df |> select(country, year, starts_with("gdp"))
df |> select(iso3 = country_code, year)    # keep and rename
df |> rename(iso3 = country_code)         # rename only
```

Helpers that save real time: `starts_with()`, `ends_with()`, `contains()`, `where(is.numeric)`.

Takeaway

`select()` also sets the column order — put your ID columns first and every later step gets easier to read.

mutate(): making and fixing columns

```
df |>
  mutate(
    gdp_pc = gdp / pop,
    log_gdp = log(gdp),
    gdp     = na_if(gdp, -99),
    large   = if_else(pop > 50, "large", "small"),
    bloc     = case_when(
      region == "Europe" & eu == 1 ~ "EU",
      region == "Europe"           ~ "non-EU Europe",
      .default                     = "other"
    )
  )
```

- `na_if()` converts a sentinel value to real NA
- `if_else()` for two outcomes, `case_when()` for more
- `mutate()` can use a column it just created in the same call

group_by() and summarize()

```
df |>
  group_by(region, year) |>
  summarize(
    mean_gdp = mean(gdp, na.rm = TRUE),
    n        = n(),
    .groups  = "drop"
  )
```

- group_by() changes nothing on its own; it marks the data
- summarize() collapses each group to one row
- mutate() on grouped data keeps all rows — that is how you get group means and shares
- count(df, region) is shorthand for group-and-tally

Takeaway

Always .groups = "drop" or ungroup() when done. Silent grouping causes wrong answers three steps later.

Joins

```
left_join(x, y, by = "iso3")    # keep all of x
inner_join(x, y, by = "iso3")   # keep matches only
full_join(x, y, by = "iso3")    # keep everything
anti_join(x, y, by = "iso3")    # rows of x with no match -- diagnostic gold
```

For panel data the key is usually two columns:

```
left_join(x, y, by = join_by(iso3, year))
```

Takeaway

Country names are a notoriously bad key. "Korea, Rep." and "South Korea" will never match. Use ISO codes.

A join you can trust

```
nrow(clean)

anti_join(clean, regimes, by = join_by(iso3, year)) |> count(iso3)

panel <- clean |>
  left_join(regimes, by = join_by(iso3, year),
            relationship = "one-to-one")

stopifnot(nrow(panel) == nrow(clean))
```

1. Note the row count **before**
2. `anti_join()` first — see what will fail to match, and why
3. State the relationship you expect; R errors if it is violated
4. Assert the row count **after**

Takeaway

If the row count grew, your key is not unique and you have silently duplicated data.

Three rules:

1. Each variable is a column
2. Each observation is a row
3. Each type of observational unit is a table

Untidy, and very common:

country	gdp_2020	gdp_2021
USA	21.0	23.3
JPN	5.0	4.9

Takeaway

Year is hiding in the column names.


```
long <- wide |>
  pivot_longer(
    cols           = starts_with("gdp_"),
    names_to       = "year",
    names_prefix   = "gdp_",
    names_transform = list(year = as.integer),
    values_to      = "gdp"
  )

wide_again <- long |>
  pivot_wider(names_from = year, values_from = gdp)
```

Takeaway

Long form is what ggplot2 and most models want. Wide form is what humans want in a printed table. You will move back and forth constantly.

Missing values

```
NA > 5                # NA
mean(c(1, NA))        # NA
mean(c(1, NA), na.rm = TRUE) # 1

is.na(x)              # test for NA -- never use x == NA
sum(is.na(df$gdp))    # count them
df |> filter(!is.na(gdp))
df |> drop_na()       # drops any row with any NA -- use knowingly
```

Takeaway

NA is contagious by design: R refuses to guess. Before dropping rows, ask whether missingness is related to your outcome — that question is substantive, not technical.

Putting it together

```
library(tidyverse)

raw <- read_csv("data/raw/wdi.csv", na = c("", "..", "-99"))
glimpse(raw)

clean <- raw |>
  rename(iso3 = country_code) |>
  filter(year >= 1990) |>
  mutate(gdp_pc = gdp / pop) |>
  select(iso3, region, year, gdp_pc, pop)

regimes <- read_rds("data/raw/vdem.rds") |>
  select(iso3, year, regime)

panel <- clean |>
  left_join(regimes, by = join_by(iso3, year),
            relationship = "one-to-one")

stopifnot(nrow(panel) == nrow(clean))

write_rds(panel, "data/clean/panel.rds")
```

When you are stuck

1. Read the error message. All of it, slowly.
2. `?function_name` — go straight to **Arguments** and **Examples**
3. Check the obvious: is the object spelled right, is the package loaded, is the column the type you think it is
4. `glimpse()` the object at the step where it broke
5. Search the exact error text, minus your variable names
6. Ask. Bring the code and the full message.

Takeaway

Getting unstuck is a skill with a method. It is not a talent you either have or don't.

Takeaway

Monday: probability, then visualization and Quarto.